Reading Model Capacity From Held-Out Data: One Instrument for Component Count, Network Depth, and Noise Structure

Jordan Elridge

10 July 2026

Summary

A recurring question in probabilistic modelling is *how much capacity a given input deserves*: how many mixture components its predictive distribution needs, how many network layers its prediction requires, how flexible its noise model must be. The obstacle is that the natural thing to optimise — the fit on the training data — is useless for answering it: training fit only ever improves with more capacity, so a model that chooses its own capacity from the training objective always chooses the most. This report shows that a single instrument answers all three questions and answers them honestly. Train a model whose capacity is a *nested* ladder — every smaller setting a valid sub-model of every larger one (for the noise model, an ordered family of increasing flexibility) — and then read the capacity each input needs from a **separate, held-out** sample rather than from the training objective. The single unifying element is that held-out reader: the same read selects the number of mixture components in a probabilistic regressor, the number of layers in a depth-adaptive network, and the flexibility of a variance function.

Three findings hold across all three settings. First, **in each setting a held-out read abstains on its control** — data built to need no extra capacity reads negative, so the instrument does not manufacture structure. How much the aggregate *knee* then resolves grades by setting: for network depth it detects the need for depth on each structured problem and abstains on the control — with the qualification, established in review, that the *selected* depth is trustworthy only when the knee is read on dedicated same-depth models — depth 2 on both structured problems, coherently across seeds on the first and marginally on the second — not on the nested ladder itself, whose reading is inflated by the cost of nesting (Section 4); for noise flexibility it correctly detects that the noise varies and abstains on the constant-noise control, but it is a *coarse* selector — it stops at a rough, stepwise noise model even though smoother rungs demonstrably recover the true noise function better (Section 5); and for component count it under-reports even the *presence* of structure, abstaining on genuinely multi-component data as well as on controls — so there the discriminating quantity is instead the size of the *local advantage*, large where components are truly needed and near-zero on the single-component controls. Second, the **per-input** version of the read is far more *data-hungry* — recovering how capacity varies from one input to the next needs much more held-out data per input than the aggregate read needs in total — and how much data rescues it depends on the setting. The per-input component count comes through cleanly on fixed-mode structure, and a sample-size sweep recovers even the harder moving-mode count that failed at first, so that failure was under-powering rather than unidentifiability — though the count read survives only about one nuisance input dimension before it degrades. The per-input depth signal, by contrast, is not established at any sample size

tried, up to sixteen times the base: its signal stays flat while its detection floor shrinks and a structure-free control crosses alongside the real problem. The per-input noise structure resolves only in aggregate. Third — and this is the methodological point — the obvious reader, the point at which held-out fit stops improving (the knee), is **unfaithful**: it systematically mis-reports the finer structure. This shows up per-input in every setting, and at the aggregate level in every setting too, each with its own signature: for component count the knee abstains on data that genuinely needs two or three components; for noise it detects that the noise varies but stops short of the classes that recover the noise better; and for depth, read on the nested ladder, it *over*-reports — selecting one rung too deep on one problem, and inflating the other problem's first increment thirteen-fold — because each increment of the ladder inherits the difference in nesting cost between adjacent rungs. The faithful read is a different quantity in each setting (a local advantage score for component count, a dedicated same-capacity comparison for depth, a direct noise-recovery error for the variance model), and recovering it is what makes the capacity picture legible. Beyond measuring capacity, the read also *deploys*: distilled into a small selector — one that imitates the faithful per-input reads, a recipe a direct held-out objective ties but does not beat — it matches or beats a single global count on every case and, run end to end on the actual model, beats that model's jointly-trained form on the staircase while never underperforming a fixed oracle count on the control. The one setting where the per-input read does not merely want more data is depth: a depth requirement that is provable by construction turns out to be representable but not learnable by gradient descent, so the depth lane has no learnable positive control — an open edge, not a resolved recovery. The closest published method, hierarchical stacking (Yao et al. 2022), reads input-dependent *averaging weights* over a fixed set of models; what is new here is reading an input-dependent *capacity* over a nested ladder, together with the faithfulness and power results that qualify it.

1. The shared failure: in-sample fit cannot choose capacity

1.1 The Occam race

Fix a family of models indexed by a **capacity** $c \in \{1, \dots, C\}$ — larger c means more mixture components, more layers, or a more flexible noise function. Let $\ell_{\text{in}}(c)$ be the fit of the best capacity-c model on the training data, measured as log-likelihood (the log-probability the model assigns to the data it was trained on; higher is a better fit). The difficulty is a structural one:

$$\ell_{\text{in}}(c) \text{ is non-decreasing in } c. \tag{1}$$

A larger model contains the smaller one as a special case, so it can always match the smaller model's training fit and usually beats it by absorbing noise. Concretely, each extra free parameter buys, on average, about half a nat — a natural-log unit of log-likelihood — of training fit. The accounting is elementary: a fitted parameter absorbs, on average, about one unit of squared standardised noise from the training data (the classical price of one degree of freedom), and the Gaussian log-density pays for error through a $-\frac{1}{2}(\text{standardised error})^2$ term — so absorbing one unit of squared noise is worth half a nat of training log-likelihood. This is the same accounting that underlies the classical model-selection criteria that charge per parameter. (Appendix A works out the closely related variance-bias version exactly for a linear mean.) So the training objective, read alone, always prefers the largest capacity on offer. It cannot select.

A natural idea is to add a *penalty* that charges for capacity — a prior on the mixture weights, say, that prefers few components. This does not rescue the situation, and it is worth seeing why, because the reason motivates

the whole method, and it is an asymmetry between two quantities. The overfitting gain the prior must cancel **grows with capacity**: by the half-a-nat-per-parameter accounting above, every spurious parameter a larger model spends against the training noise buys it roughly half a nat of in-sample fit, so the gain climbs steadily as capacity — and with it the parameter count — rises. The prior's charge does **not** keep pace: a prior on the weights adds a single log-density term whose size is set by the prior's shape, not by how many parameters overfit, and — unlike the data log-likelihood — it does not accumulate over examples, so it does not grow with the amount of data. A charge that stays put cannot cancel a gain that climbs with every added component. Sweeping the prior's strength only slides that fixed charge up or down; it cannot bend to track the growing gain, so the held-out selection it implies stays essentially flat and the prior loses at every setting. This is not a tuning failure to be swept away with more settings; it is a mismatch built into the shapes of the two quantities.

The consequence is the organising principle of this report:

> The charge for capacity must come from **fresh, held-out data**, not from the training objective and not from any fixed in-sample penalty.

Held-out fit, unlike training fit, is not monotone in capacity (Figure 1). A model too large for the data pays on held-out examples, because the extra capacity was spent fitting training noise that does not recur. The capacity the data actually supports is the one that maximises held-out fit — and that is a quantity we can measure.

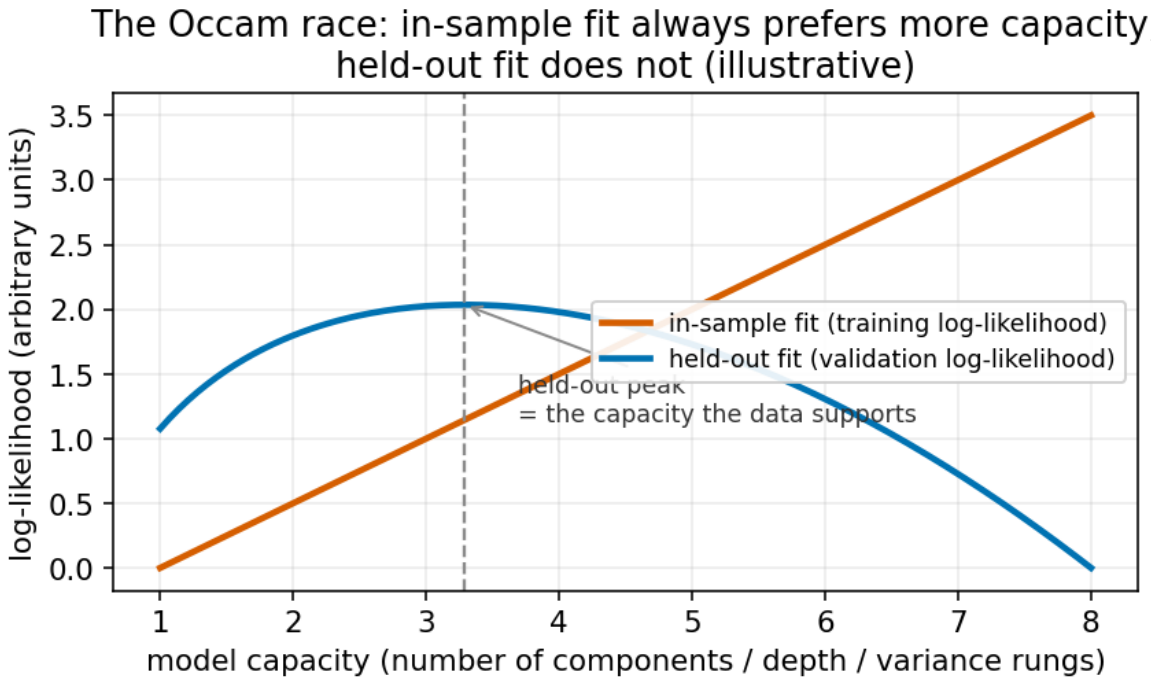

Figure 1: The Occam race, illustrated. Training fit rises monotonically with capacity (each added degree of freedom buys about half a nat on average). Held-out fit rises, peaks at the capacity the data supports, then falls as further capacity fits only training noise. The training objective alone cannot locate the peak; held-out data can. This panel is schematic — a picture of the mechanism, not a measured run.

1.2 A worked instance of the same failure: variance collapse

The clearest closed-form instance of the Occam race is the estimation of a noise variance jointly with a mean. Suppose we fit a model that predicts both a mean $\mu(x)$ and a noise variance σ^2 by maximising the Gaussian log-likelihood on the training set. Writing N for the number of training points and p for the number of parameters the mean function spends, the maximum-likelihood variance is biased *low* by an exactly known factor:

$$\mathbb{E}\left[\hat{\sigma}^2\right] \;=\; \frac{N-p}{N}\,\sigma^2. \tag{2}$$

(The derivation is in Appendix A; it is the multivariate version of the familiar "divide by $N-1$, not N" correction.) The mean function fits part of the noise, the residuals it leaves behind are too small, and the variance read off those residuals is therefore too small — the model reports itself as more certain than it is. With a *flexible* mean the effect is far larger than the linear formula, because a flexible mean drives the training residuals toward zero, and the fitted variance follows them down. This is *variance collapse*, and it is the same disease as the Occam race: the in-sample objective rewards spending capacity (here, mean flexibility) against the noise, and the price is paid out of sample. Recent work names the same pathology and proposes training-time fixes (Stirn et al. 2023); the read-out view taken here is complementary — measure the honest variance on held-out residuals rather than in-sample ones.

2. The method: a nested ladder read from held-out data

The method has one training idea and three reading idioms. It changes only how a model is trained and scored; it does not change any model architecture, and it adds no capacity penalty.

2.1 The nested capacity ladder

Train a single model whose capacity is **nested**: the capacity-c setting uses the first c of an ordered set of ingredients (the first c mixture components, the first c layers, the first c of an ordered set of noise-model rungs), and — crucially — every prefix is itself a fully trained, valid model. This is achieved by drawing a capacity at random on each training step and using only the first c ingredients on that step, so that after training every prefix has been optimised. The ingredients come out *ordered by importance*, first the most useful, because the smaller prefixes are trained more often and must stand alone. This ordered-prefix training is the nested-dropout scheme of Rippel, Gelbart, and Adams (2014), originally introduced to order the hidden units of an autoencoder; here it orders the units of a *capacity* axis. Training makes every capacity **available**; it does not select one.

2.2 The held-out score table

Freeze the trained model. On a held-out sample — data not used in training — record, for each held-out example i and each capacity c, the log-probability the capacity-c sub-model assigns to that example's true target:

$$s_i(c) \;=\; \log p_c\left(y_i \mid x_i\right). \tag{3}$$

Everything downstream touches only this table $\{s_i(c)\}$. Because the examples are held out, a capacity that overfits scores *badly* here — the Occam charge is levied automatically by the held-out fit, with no penalty term.

2.3 Reading idiom 1 — the global weights (stacking)

To ask how much capacity the *whole* dataset supports, find the single mixture of sub-models that predicts held-out data best. Fit weights $\pi = (\pi_1, \ldots, \pi_C)$ on the probability simplex ($\pi_c \geq 0$, $\sum_c \pi_c = 1$) by maximising the held-out log-score of the mixture:

$$\hat{\pi} \;=\; \arg\max_{\pi} \sum_i \log \sum_c \pi_c \, \exp\big(s_i(c)\big). \tag{4}$$

This is *stacking* of predictive distributions (Yao et al. 2018). It is a concave problem solved in seconds, and it is the right thing to optimise because the log-score is a *proper* scoring rule — it is maximised in expectation by the true predictive distribution, so weights that predict held-out data well are weights that describe the data-generating process well. Over-large capacities earn near-zero weight because they predict held-out data poorly. (Appendix B gives the objective and its optimiser.)

2.4 Reading idiom 2 — the knee, and its guardrails

Stacking returns soft weights; often we want a single answer, "how many does this dataset need?" Define the **increment curve** as the extra held-out fit each capacity buys over the smallest, where the held-out fit at capacity c is the total held-out log-score $\sum_i s_i(c)$:

$$\Delta(c) \;=\; \sum_i s_i(c) \;-\; \sum_i s_i(1). \tag{5}$$

So $\Delta(c)$ is *cumulative* — the total gain of capacity c over capacity 1 — whereas the stopping test below reads the *successive* increment $\Delta(c) - \Delta(c-1)$, the gain of one more rung. Walk c upward from $c = 1$ and **accept** each capacity c whose increment over the previous rung, $\Delta(c) - \Delta(c-1)$, is *statistically significant* — larger than twice its standard error, the error being computed by resampling the held-out examples (a plain bootstrap over independent examples; see the guardrails below). Stop at the first rung whose increment fails that test. The **selected capacity** r^* — the **knee** of the held-out-fit curve, where extra capacity stops paying — is the largest capacity accepted (one below the first rung that fails, or $r^* = 0$ if the very first rung tested, capacity 2, already fails). When we later say "the knee reads r^*" this is the rule meant. If even capacity 2 fails — the $1 \to 2$ increment does not clear the bar — no capacity beyond the base is supported and the reader **abstains**, which we write $r^* = 0$ (a flag, distinct from the counts $2, 3, \ldots$ the rule otherwise returns; this global knee never returns 1, because reaching capacity 2 is what the first test decides — though the *per-input* knee of Table 2 reports raw component counts and so does show 1). This abstention is not a failure mode; it is the reader's negative answer, and it is the single most important thing to check, because a reader that never abstains is a reader that hallucinates structure.

Four guardrails make the read trustworthy.

- **The negative control.** For each setting we build **negative-control twins** — datasets with the same gross statistics but no genuine capacity structure, a mixture whose components all coincide or a problem whose

complexity is uniform across inputs. On a twin the reader must abstain. A twin that reads positive would tell us the instrument fabricates structure.

- **The coherence check.** A nested prefix might be a *worse* model of capacity c than a network trained from scratch at exactly capacity c, because the prefix shares parameters with larger capacities. We measure this **coherence cost** directly — held-out fit of the nested rung minus that of a dedicated same-size model, so a genuine cost of nesting shows as a *negative* number (and a benefit as positive) — so any cost of nesting is reported, not hidden. This check turned out to be more than an honesty item: where the coherence cost *differs between adjacent rungs*, the difference leaks into the ladder's increment curve and biases the knee itself (Sections 3 and 4 each exhibit one direction of this bias).
- **Independent-example error bars, paired across rungs.** The bootstrap that sizes the significance test resamples *individual held-out examples*, which are independent, and computes both adjacent rungs' scores on the *same* resampled examples — so the strong correlation between neighbouring rungs' scores (the same hard example hurts both) is built into the error bar rather than ignored. Resampling coarser groups (for instance by random seed) inflates the error bars and makes the reader abstain spuriously; we learned this the hard way and fixed it everywhere.
- **Small-gap honesty.** When the total held-out gain at stake is a few thousandths of a nat per example, *any* estimate of its standard error — bootstrap included — is itself unreliable (Sivula et al. 2020), and the error bars capture only the sampling variability of the held-out set, not run-to-run training variability (which is why every verdict here requires three independent training runs). A near-flat increment in that regime is therefore reported as "unresolvable at this sample size", never as a clean negative.

2.5 Reading idiom 3 — the per-input read, and the faithful version of it

To ask how capacity varies *from one input to the next*, bin the held-out data by a cheap observable feature of the input (its region, its local density) and run the stacking of idiom 1 *separately in each bin*. The per-input answer is then "look up which bin the input falls in, use that bin's weights." This is in the spirit of *hierarchical stacking* (Yao et al. 2022), which makes the stacking weights depend on the input — with one deliberate simplification: in hierarchical stacking the per-bin weights are *partially pooled*, shrunk toward the global weights by a hierarchical prior so that data-poor bins borrow strength from the whole, whereas here each bin is fit independently. The deliberate low resolution of the binning — a handful of numbers, not a free function of the input — is the **safety mechanism**: a weight map with little capacity cannot memorise the held-out set, so it cannot re-introduce the in-sample overfitting the whole method exists to avoid. (Partial pooling is the natural next refinement, precisely because the per-input read turns out to be limited by the data inside each bin — Section 4.)

There is a subtlety that took all three studies to see clearly. The obvious per-input reader — run the knee of idiom 2 inside each small neighbourhood — is **unfaithful**: with few points per neighbourhood its significance test is noisy and it *under-reports*, stopping early. The faithful per-input quantity is not the knee but a smoother contrast. For component count we use the **local advantage**: in a neighbourhood of an input (a sliding window spanning about 7.5% of the input range), the held-out fit of the top capacity rung minus the held-out fit of the single-component model, averaged over neighbours. This advantage rises smoothly where inputs genuinely need more components and stays near zero where they do not, and it recovers the per-input structure that the per-input knee misses. For the variance model the faithful quantity is even more direct — the **noise-recovery error**, the error in the recovered noise level against the known truth. Concretely it is the average over inputs of $\left|\hat{\sigma}(x)/\sigma(x) - 1\right|$, the relative error of the recovered noise scale (zero when the model reads the true noise everywhere), as Section 5 shows.

2.6 Estimating global nuisances honestly

One quantity — the overall noise level — is not a capacity to be selected but a nuisance to be estimated, and Section 1.2 showed the in-sample estimate is biased. Two held-out estimators fix it. The **cross-fitted residual** estimator splits the data into folds, fits the mean on all folds but one, and measures residual variance on the held-out fold, so the variance is never read off residuals the mean was fitted to. The **marginal-likelihood** estimator, available for a linear mean, estimates the noise level from the probability the model assigns to the data after integrating the mean parameters out; integrating over the means charges for the volume of mean functions that fit the data, which supplies the same degrees-of-freedom correction as Appendix A automatically. Both are unbiased when the mean is well specified; the cross-fitted estimator degrades gracefully when it is not, provided the mean is trained to convergence.

3. Component count in a probabilistic regressor

The first setting is a regressor that predicts a mixture distribution — several candidate means each with its own variance, combined by weights — where the number of components a given input needs varies across inputs. The capacity axis is the number of components c; the ladder offers more rungs than any problem needs — up to eight components on the three-region staircase (whose true count reaches three) and up to six on the two-component problems (whose true count reaches two) — so there is always headroom above the correct count. The pre-registered question: does the nested ladder, read on held-out data, recover the per-input component count, and does it abstain where it should?

Test problems. Table 1 lists them. The load-bearing one is toy D, a *staircase*: the true per-input component count, written k^*, rises across three input regions, $k^* = 1 \to 2 \to 3$ (distinct from the tested capacity c of Section 1.1). A simpler sibling, toy C, is a two-component problem whose modes separate as the input grows, so k^* rises once, $1 \to 2$. Its harder counterpart toy E is a *moving-mode* problem where the count rises then falls, $1 \to 2 \to 1$. The two-component problems C and E each have a **broad twin** — the same marginal distribution but a single genuine component throughout — as the negative control. Each problem is fit on three random seeds, giving nine structured cases and six control cases.

Table 1: The test problems across the three settings. The first four rows concern component count (Section 3) — three structured problems C, D, E with a broad-twin control built for the two-component problems C and E — the next three network depth (Section 4), the last two noise flexibility (Section 5). Each setting includes a negative control (the broad twins, toy G-flat, and the homoscedastic twin respectively) that must read negative.

Problem	Capacity axis	Structure	Role
Toy C	component count	two components separating across input, count $1 \to 2$	positive test
Toy D	component count	staircase count $1 \to 2 \to 3$ across input	positive test
Toy E	component count	moving mode, count $1 \to 2 \to 1$	hard positive test
Broad twins (C, E)	component count	single component throughout	negative control

Problem	Capacity axis	Structure	Role
Toy G	network depth	required depth rises across input	positive test
Toy G-flat	network depth	uniform, base-level required depth	negative control
Toy H	network depth	noise level varies across input	signal-to-noise dial
Problem V1	noise flexibility	smooth $\sigma(x) = 0.1 + 0.3\,\mathrm{sigmoid}(4x)$	positive test
V1 homoscedastic twin	noise flexibility	constant σ	negative control

The staircase is recovered — by the faithful reader. Figure 2 shows the local advantage (Section 2.5) by input region on toy D, for each of three random seeds, against an independent **reference instrument** — a different, independently computed estimator of the same per-input count (a variational mixture fit separately within each input region), which computes the same top-versus-single-component advantage on a *separately trained* model, used here only as an external cross-check that the read is not an artefact of this ladder. On every seed the advantage rises monotonically across the three regions, tracking the true staircase. Table 2 gives the numbers, and next to them the *per-input knee* — the unfaithful reader — for contrast.

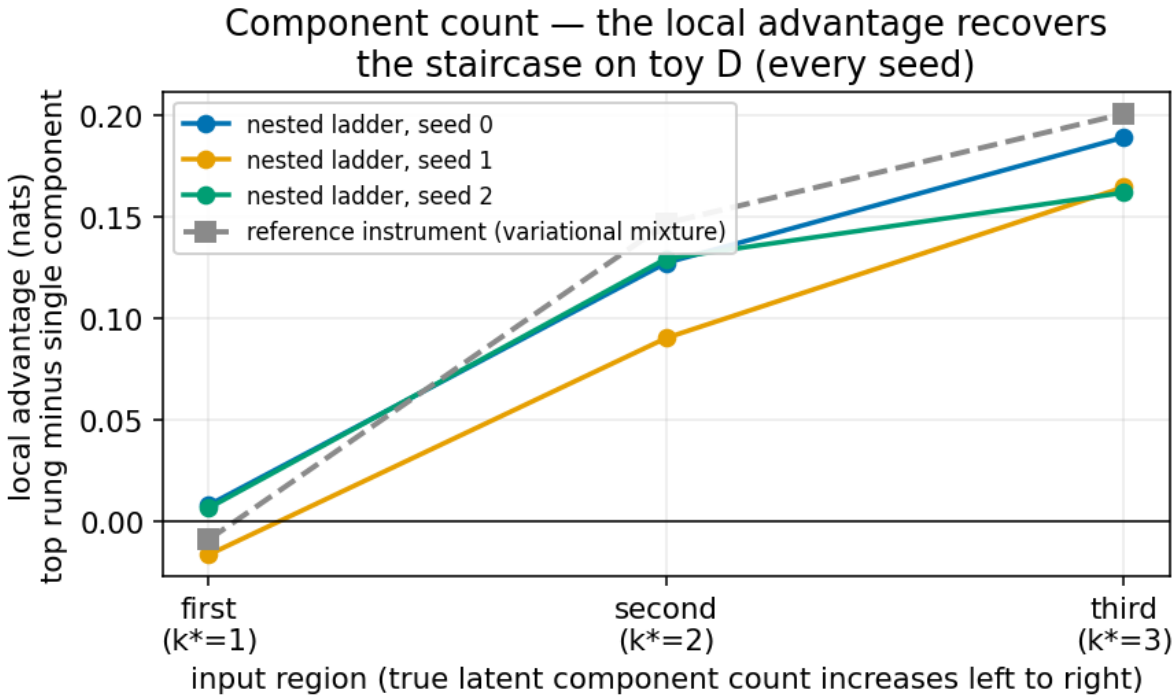

Figure 2: On toy D the local advantage (held-out fit of the top rung minus the single-component model, neighbour-averaged) rises across the three input regions on every seed, tracking the true count $1 \to 2 \to 3$ and matching an independent reference instrument. This is the faithful per-input read.

Table 2: Toy D, the staircase. The local advantage (in nats, the natural-log units of log-likelihood) rises with the true count on every seed and matches the reference. The per-input knee, by contrast, is erratic across seeds and does not track the staircase — it is the unfaithful reader. (The per-input knee is reported as an actual component count, so a single-component read shows as 1 — the same base outcome the global knee flags as $r^* = 0$ in Section 2.4.)

Read	Region 1 (k^*=1)	Region 2 (k^*=2)	Region 3 (k^*=3)
Local advantage, seed 0	0.008	0.127	0.189
Local advantage, seed 1	-0.016	0.090	0.165
Local advantage, seed 2	0.007	0.130	0.162
Reference instrument	-0.009	0.147	0.201
Per-input knee, seed 0	1	2	4
Per-input knee, seed 1	1	1	1
Per-input knee, seed 2	1	1	2

The contrast in Table 2 is the central methodological result of this section. The faithful reader (local advantage) recovers the staircase on all three seeds; the per-input knee, run on the identical data, gives $\{1, 2, 4\}$, $\{1, 1, 1\}$, $\{1, 1, 2\}$ — erratic across seeds: it rises correctly in the first two regions on seed 0 but overshoots to four in the top region, and collapses to all-ones on seed 1, so it does not reliably track the staircase. The knee is noisy precisely because each neighbourhood holds few points; the advantage is smooth because it averages a contrast rather than testing a sequence of increments.

The simpler positive test recovers too. Toy C — the fixed $1 \to 2$ case — is recovered on all three seeds by the same local advantage (Table 3): averaged over each region it stays near zero where a single component suffices and rises to a clear positive where the second component resolves. It is the easy companion to the staircase, confirming the reader on a clean fixed-mode step; toy D is the harder, load-bearing case shown in detail above.

The controls abstain cleanly. On the six broad-twin control cases (two problems, three seeds), the global knee abstains ($r^* = 0$) in every case, and the largest local advantage anywhere is 0.016 nats (Table 3) — an order of magnitude below the 0.16–0.20 seen on the structured staircase. The instrument does not fire on single-component data.

The moving mode is not recovered — an honest negative, with a sharpened suspect. Toy E, whose count rises then falls, is recovered on only one of three seeds (Table 3); on the other two the read is flat or inverted, and the per-region reads even disagree across seeds — the correlation between two seeds' region-by-region local-advantage profiles falls as low as -0.838. The positive claim is therefore scoped to *fixed-mode* structure (the -0.838 is a correlation over just three region means — reported for its sign, not its precision). One further piece of evidence localises the difficulty: the same independent reference-instrument method whose staircase read appears in Table 2 — a variational mixture fit separately within each input region, with no nesting — *did* recover this rising-then-falling count on its documented (single) fit on this problem, reading $-0.018 / +0.149 / -0.026$ across the three regions: near zero at the single-component ends, clearly positive at the two-component middle. That single documented fit was the basis for a natural suspicion — that the difficulty lived in the nested training rather than in the per-input read as such, since nesting imposes one *global* importance ordering on

the components while a count that rises then falls wants a different ordering in different input regions. The suspicion did not survive testing. Running the non-nested reader across seeds on the identical data recovered the hump on only one seed as well (with its own control flat), and a multi-restart version confirmed that the one non-recovered seed is genuinely absent at this sample size rather than an optimisation artifact — so the difficulty is *not* specific to nesting. The real resolution is simpler and is deferred to Section 6.4: a sample-size sweep recovers the moving-mode count with more held-out data — two seeds at this sample size, all three at four times the data, the control flat throughout — so the moving mode is *under-powered here, not unidentifiable*. At the sample size of this section it is recovered on one seed of three; the power curve that lifts it to all three is in Section 6.4.

Table 3: The component-count cases beyond toy D (which is detailed in Table 2). The local advantage is averaged over the region where the true count is one and the region where it is two (for the moving-mode toy E, the k^*=2 region is the central hump). It rises to a clear positive in the two-component region on toy C for every seed and on toy E for seed 0 only — the moving-mode failure — while on the single-component controls it never exceeds $+0.016$ anywhere. The global knee abstains ($r^* = 0$) on all of these, *including the genuinely structured C and E* — the same under-reporting the per-input knee shows on toy D in Table 2. It is the local advantage, not the knee, that separates structure from control here.

Component-count case	True structure	Local advantage, 1-component region (per seed)	Local advantage, 2-component region (per seed)	Global knee r^*
Toy C	step $1 \to 2$	-0.025, -0.016, $+0.004$	$+0.061$, $+0.037$, $+0.048$	0, 0, 0
Toy E	moving $1 \to 2 \to 1$	$+0.007$, $+0.006$, $+0.004$	$+0.075$, $+0.000$, -0.005	0, 0, 0
Broad twins (control)	single component	—	$\leq +0.016$ (max anywhere)	0, 0, 0

The staircase itself completes this picture, and supplies the mechanism. On toy D — where the local advantage recovers the full staircase on every seed — the *global* knee reads 2 on one seed and abstains on the other two (per-seed $r^* = 2, 0, 0$). This global-knee irreproducibility was pre-registered as a decisive check and is reconciled here rather than in the original adjudication: the knee's walk stalls exactly at the middle rungs where the coherence cost of nesting concentrates (Section 3's cost paragraph below). The mechanism is one line of algebra. Write $\kappa(c)$ for the coherence cost of rung c — the held-out fit of the nested rung minus that of a dedicated capacity-c model (Section 2.4) — so that $\text{nested}(c) = \text{dedicated}(c) + \kappa(c)$. Then every increment the knee tests decomposes as

$$\underbrace{\text{nested}(c{+}1) - \text{nested}(c)}_{\text{what the knee tests}} = \underbrace{\text{dedicated}(c{+}1) - \text{dedicated}(c)}_{\text{true gain of one more rung}} + \underbrace{\kappa(c{+}1) - \kappa(c)}_{\text{change in nesting cost}} . \tag{6}$$

Where the cost peaks in the middle, the middle increments are dragged below their significance bar and the walk stops early. The knee read off a nested ladder, in other words, is only as faithful as the ladder is uniformly coherent — a point Section 4 meets again from the opposite direction.

Nesting has a small, localised cost. The coherence cost (Section 2.4) — nested rung minus a dedicated same-size model — ranges from -0.126 to $+0.089$ nats across the structured cases. The "worse by more than two standard errors" flag does fire on eight of nine cases, but always on a *middle* transition rung and never on the top rung the read actually uses — so the cost is small in magnitude and confined to rungs the reader does not depend on. The largest cost (-0.126 nats, on the third rung of one seed) sits on a case whose staircase the advantage still recovers. Nesting is not free, but its cost does not break the read.

The per-input read distils into a deployable router. A small map from input to component count, trained on the held-out reads with the ladder frozen, recovers a deployable per-input policy. Table 4 reports it. Trained on the *smooth* per-input responsibilities (the posterior weight each component takes for each point) it beats or ties a single global count on all nine cases; trained instead on the *hard* knee labels it is worse than the smooth-trained router on seven of nine, and a third variant trained on *raw per-example labels* — each held-out point's own best capacity, with no smoothing — is worse still, on eight of nine. This is a direct downstream confirmation that the smooth read is the faithful one and the knee is not.

Table 4: The distilled router, scored by held-out negative log-likelihood — the negative of the held-out fit used elsewhere in this report, so here *lower* is better. The smooth-responsibility router is the reference; it is the only one benchmarked against a single global count (which it beats or ties on all nine cases), and the hard-knee and raw-label routers are each benchmarked against *it*. Both are worse, confirming the knee's unfaithfulness downstream. The three number pairs in the first row are the router's versus the global count's held-out negative log-likelihood on the staircase problem's three seeds (router first, lower is better).

Router training target	Outcome	Compared against
Smooth per-input responsibilities	beats or ties on 9 of 9 (staircase wins 0.856 vs 0.885, 0.860 vs 0.949, 0.826 vs 0.922)	a single global count
Hard knee labels	worse on 7 of 9	the smooth-responsibility router
Raw per-example labels	worse on 8 of 9	the smooth-responsibility router

4. Network depth

The second setting is a network that can use a variable number of layers, where the depth an input needs varies across inputs; the capacity axis is the number of active layers. Toy G needs more depth on some inputs than others; toy G-flat needs a uniform depth (the control); toy H holds its mean function fixed (a shape that

needs about depth 2 globally) but varies its noise level across inputs, a signal-to-noise "dial" whose per-input question is whether the reader spends less depth where the noise is higher. The ladder is a single depth-nested network, trained with the ordered-prefix scheme, and read on a held-out sample of 500 points per seed.

The aggregate read: detection is clean; the selected depth needs a dedicated comparison. Figure 3 shows the global increment curve $\Delta(c)$ over depth for the three problems, as read off the nested ladder. On toy G it climbs through depth 3 and flattens (the knee of idiom 2 selects depth $r^* = 3$, while the soft stacking weights of idiom 1 spread over depths 3 and 4 — the two reading idioms, not a contradiction); on toy H it selects depth 2; on the control toy G-flat the very first increment is already non-significant and the curve is flat-to-negative, so the reader **abstains** ($r^* = 0$) on every seed. The instrument separates the two problems that need depth from the control that does not.

That is the *detection* result, and it stands. The *selected depths*, however, inherit a bias this study only quantified in review. The coherence cost of nesting (measured below) is not uniform across depth: it is by far largest at depth 1 and decays as depth grows. By the decomposition displayed in Section 3, each increment of the nested curve is the true gain of one more layer *plus the change in nesting cost between adjacent rungs* — and a cost that shrinks with depth adds a spurious positive term to every early increment. The same experiment also trained a *dedicated* network at each fixed depth and scored it on the identical held-out examples, so the corrected read is available directly: walking the same knee rule over the dedicated models' scores (Table 5) reads **depth 2 on toy G, on every seed and pooled** — one rung below the nested read, whose extra rung is accounted for almost exactly by the measured cost change — and **depth 2 on toy H pooled**, but from a first increment of $+0.049$ nats rather than the ladder's $+0.64$: thirteen-fold smaller once the nesting interference is removed, real but marginal at this sample size (one seed's read abstains). The control abstains under both instruments. This is the same lesson as the staircase's global knee in Section 3, with the opposite sign: there a mid-ladder cost suppressed increments and the knee under-read; here a shallow-rung cost inflates them and the knee over-reads. A knee walked on a nested ladder is only as faithful as the ladder is uniformly coherent; the dedicated-model comparison (or a read, like the local advantage, that contrasts against a fixed reference) is the honest aggregate instrument when it is not.

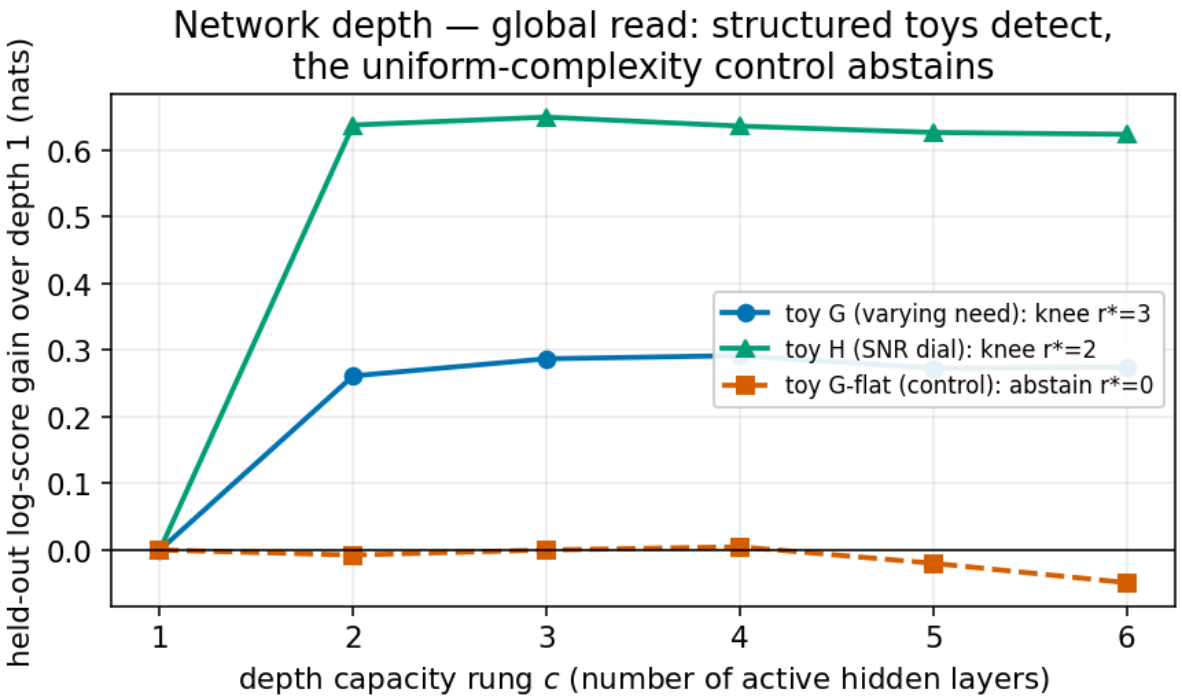

Figure 3: The global depth read, from the nested ladder. On toys G and H the held-out fit climbs with depth and this reader selects depth 3 and 2 respectively; on the uniform-complexity control G-flat the curve is flat-to-negative and the reader abstains. The negative control is the load-bearing check. The early increments of these curves carry the nesting-interference term quantified in the text; the dedicated same-depth comparison of Table 5 corrects the selected depths to 2 on both structured problems.

Table 5: The depth knee, read on the nested ladder versus on dedicated fixed-depth networks scored on the identical held-out examples (pooled over seeds; mean $\pm$ one standard error of each increment; knees per the rule of Section 2.4). The dedicated read is the honest one: toy G's nested read of 3 drops to a seed-coherent 2, toy H's dramatic first increment shrinks thirteen-fold to a real-but-marginal $+0.049$, and the control abstains either way. The difference between the two instruments' increments matches the measured change in nesting cost between adjacent rungs.

Problem	Ladder: 1st, 2nd increment	Ladder knee	Dedicated: 1st, 2nd increment	Dedicated knee
Toy G	$+0.261 \pm 0.015$, $+0.026 \pm 0.009$	3 (seeds 4, 2, 3)	$+0.126 \pm 0.012$, -0.009 ± 0.009	**2** (seeds 2, 2, 2)
Toy H	$+0.638 \pm 0.023$, $+0.012 \pm 0.009$	2 (seeds 3, 2, 2)	$+0.049 \pm 0.011$, -0.008 ± 0.010	**2** (seeds 2, 3, abstain)
Toy G-flat	-0.008 ± 0.003 (first)	abstain	-0.019 ± 0.006 (first)	abstain (all seeds)

The per-input read is at the noise floor at this sample size. Figure 4 shows the per-bin advantage over the global read — does splitting by input region beat a single global depth? — with two-standard-error bars, for each problem and seed. On toy G one seed appears to clear the bar decisively ($+0.0122 \pm 0.0046$ nats, $+2.7$ standard errors), above the ceiling set by the control (whose best is $+0.0062$ at $+1.6$ standard errors, never significant). In review, however, this one hit did not survive re-drawing the random partition that splits the held-out data into a weight-fitting half and a scoring half: across nine such partitions the standardised advantage reads $+2.7$, -0.4, $+2.6$, $+1.0$, $+0.5$, $+2.5$, 0.0, $+0.6$, -0.7 standard errors — it clears the bar on only three, and is null or negative on the rest — one favourable draw of a split-sensitive statistic, not a stable signal. Two of the eighteen (problem, seed, resolution) cells clear the nominal bar overall, which is within what chance alone would produce, and neither survives halving the bin width: toy G seed 2 falls to $+0.010 \pm 0.010$ at the finer bins, and the one finer-bin hit (toy H seed 2, $+0.040 \pm 0.015$) has no counterpart at the coarser bins. The reason is simple arithmetic: the per-input test at 500 held-out points, split roughly in half for fitting the bin weights and for scoring them and then into three bins, leaves roughly **83 points per bin** for scoring, which is badly underpowered. The honest conclusion is that no per-input depth signal is established at this sample size — the read sits at the noise floor, and settling whether the signal exists needs either more held-out data or a stronger estimator. Both were tried, and neither rescues it: a repeated-split estimator (averaging the advantage over many random partitions) and a partially-pooled version of the per-bin weights (Section 2.5) both come back null, and a sample-size sweep leaves the signal flat while its detection floor shrinks, up to sixteen times this sample — the read is not merely underpowered here but not established at any size tried (Section 6.4), for a reason Section 6.5 makes precise. Table 6 gives the per-bin numbers at this sample size.

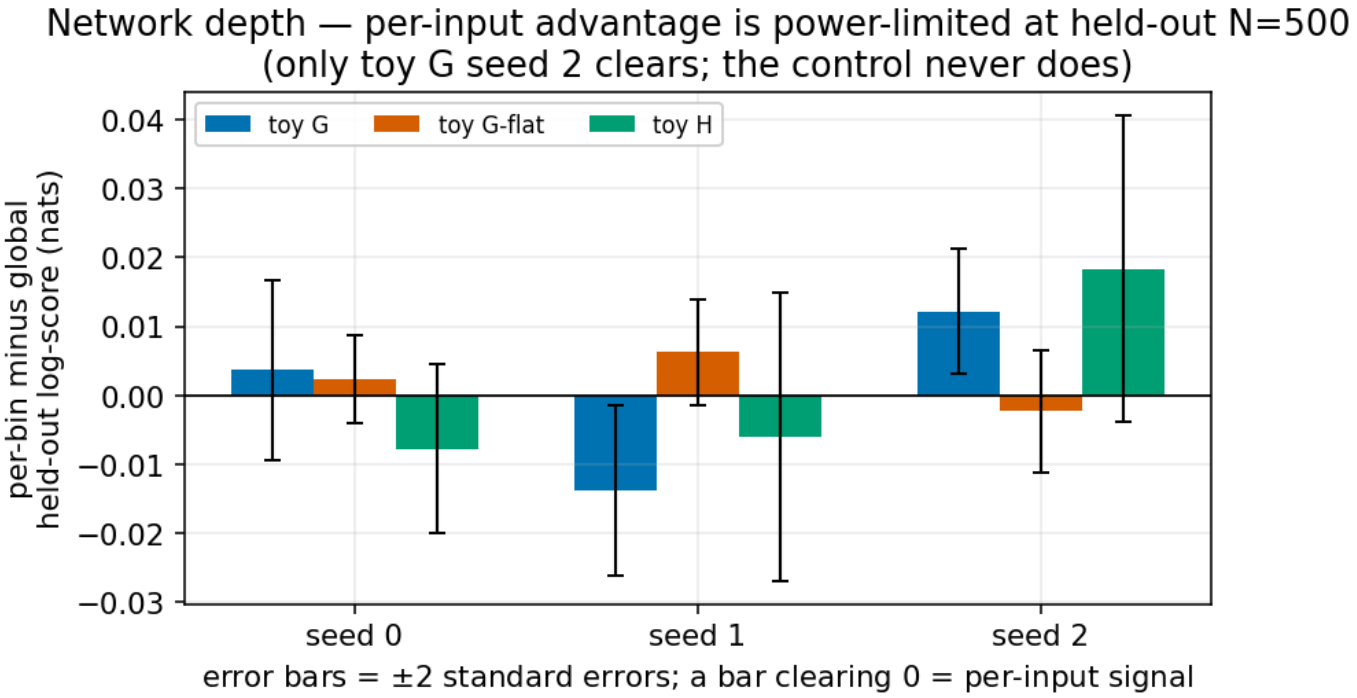

Figure 4: Per-input (per-bin) advantage over the global depth, with two-standard-error bars. Only toy G seed 2 clears zero — a hit that does not survive re-drawing the fit/score partition (see text); the control G-flat never clears (no false positive); toy H does not clear at this bin size. At roughly 83 held-out points per bin the per-input test is underpowered.

Table 6: Per-bin advantage over global depth (nats; mean ± one standard error). Bold marks the one cell clearing two standard errors — a hit that does not survive re-drawing the fit/score partition (see text). The control G-flat clears on no seed — the no-false-positive bar — and toy H clears on none at this bin size.

Problem	Seed 0	Seed 1	Seed 2
Toy G	$+0.0037 \pm 0.0066$	-0.0139 ± 0.0062	$\mathbf{+0.0122 \pm 0.0046}$
Toy G-flat (control)	$+0.0024 \pm 0.0032$	$+0.0062 \pm 0.0039$	-0.0022 ± 0.0044
Toy H	-0.0078 ± 0.0061	-0.0060 ± 0.0105	$+0.0184 \pm 0.0111$

The control never fires, and that is what makes the reads interpretable. On toy G-flat no seed beats the global read at either bin resolution, and the global knee abstains under both instruments of Table 5. The instrument invents no per-input structure on uniform-complexity data. Toy H, whose noise level varies but whose mean function does not, was pre-registered to pass only if its per-input advantage beat the global read by more than two standard errors *and* used less depth where the noise is higher. Its *selected depth* does differ between its high- and low-noise regions on two of three seeds, but the per-bin advantage (Table 6) clears the two-standard-error bar on none, and the depth-down-where-noisier direction holds on none; we report it as *not confirmed* rather than dress it up.

Nesting is materially costly on the depth ladder — far more so than for component count. The coherence check fails on roughly half of all (depth, problem, seed) cells, the worst at the shallowest rung: the nested depth-1 sub-model scores up to 0.72 nats *worse* per example than a dedicated one-layer network on toy H (about 0.13–0.21 on toy G), falling to at most 0.08 nats by depth 2 on both problems and staying small thereafter — against a worst case of 0.13 on the component ladder — and, unlike the component ladder, the cost reaches the top rung. The mechanism is plausibly the single readout layer serving all six depths, with the shallowest depth least able to claim it; the cost is nearly absent on the control (≤ 0.024 nats), whose uniform problem gives the depths nothing to compete over. This cost profile is exactly what biases the nested knee (Table 5 above), and any deployment claim must carry it. A deployable per-input depth router, the analogue of the component-count router of Section 3, remains the natural next step and is scoped to the outcome the component-count router actually delivered — a router that ties or beats a single global depth plus a compute saving from running only the selected depth per input — rather than to a per-input win on a signal this section could not establish. Section 6 pursues exactly this scoping: the per-input depth signal stays null at every sample size tried, so the compute-saving framing, not a per-input win, is the one that survives.

5. Noise structure

The third setting selects not a count but the *flexibility of the noise model*: given a regressor that predicts an input-dependent noise level $\sigma(x)$, how flexible should $\sigma(x)$ be? The capacity axis is an ordered ladder of four noise models — a single constant (labelled v0 in Figure 6), a coarse three-step function (v1), a smooth monotone function (v2), and a fully flexible network (v3) — read by the same held-out knee. (The lower-case v-rungs are capacities on this ladder — v0 the base, v1 the next step up, and so on — not to be confused with problem V1 of Table 1; in the counting of Section 2.4 a knee that abstains to v0 reads $r^* = 0$, and one that selects v1 reads $r^* = 2$.) Unlike the component and depth ladders, which are one prefix-nested model so that every prefix is

itself a trained sub-model, these four noise models are fit as a *family ordered by flexibility* — each rung at least as expressive as the constant, and the flexible network able to represent the others — so what carries over is not the shared-parameter nesting but the held-out *reading*: the same knee walking an ordered capacity axis. This setting also exhibits the variance collapse of Section 1.2 in its rawest form, so we treat both: first the collapse, then the capacity read.

The collapse is real, and it is finite-sample. Figure 5 shows a model jointly fitting a mean and a noise level on a small sample ($N = 200$). Early in training the model's reported spread sits *above* the truth; as training proceeds the mean overfits, the in-sample residuals shrink, and the reported spread falls through the truth to 0.80 of it, while the held-out standardised squared residual — the average squared prediction error divided by the model's own predicted variance, which equals 1 exactly when the noise estimate is honest — climbs from about 1.0 to 3.7. The model becomes badly overconfident, and the held-out fit is best at an early stopping point (marked) long before training ends. The same experiment at $N = 1000$ shows essentially no collapse: the disease is finite-sample, severe at small N and largely gone by moderate N.

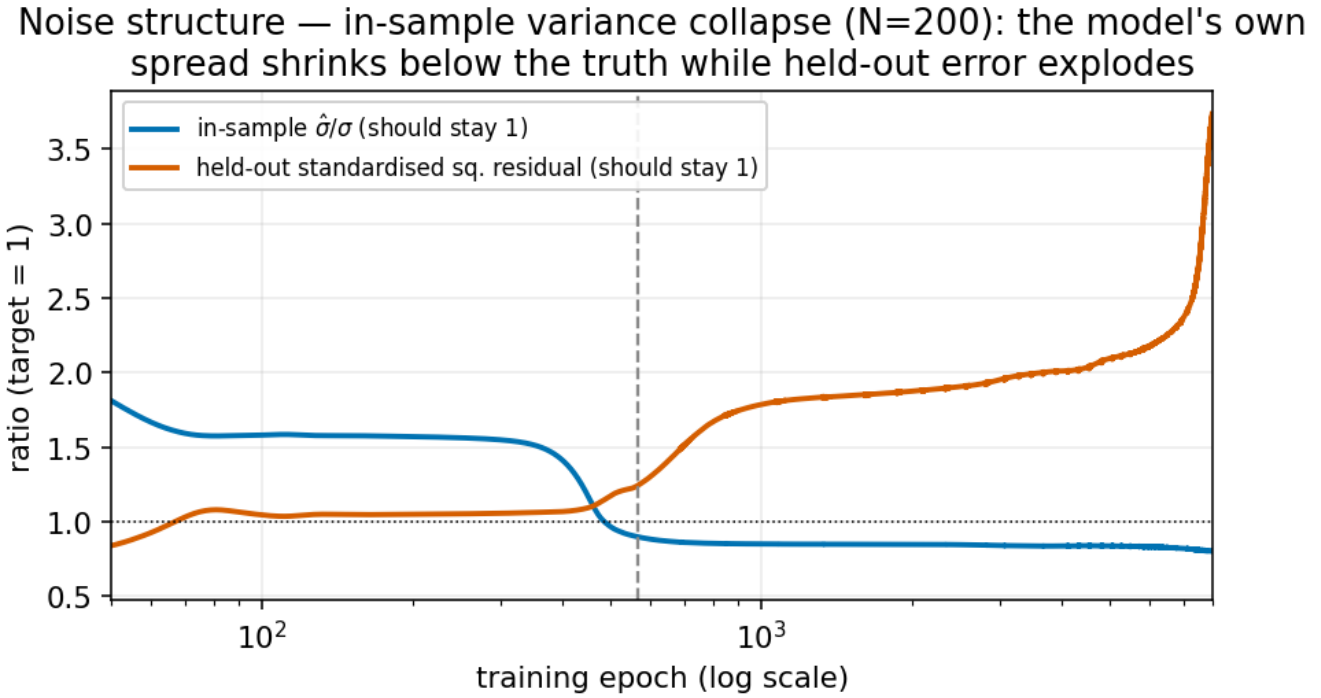

Figure 5: In-sample variance collapse on a small sample ($N = 200$). As the mean overfits, the model's own reported spread ($\hat{\sigma}/\sigma$, blue) falls below the truth while the held-out error (red) explodes far above it; both should sit at 1. Held-out fit is best at the early point marked. At $N = 1000$ the effect is essentially absent — the collapse is finite-sample.

The honest estimators recover the truth; one has a sharp condition. Estimating the noise on held-out or cross-fitted residuals (Section 2.6) removes the collapse: on well-specified problems the cross-fitted and marginal-likelihood estimators match the true variance, while the in-sample estimate is biased low. When the mean is *mis-specified*, the honest-residual estimators absorb the bias into a safe *over*-estimate rather than collapsing — the safe direction. The one sharp condition, which matters for deployment: the honest-residual estimator on a *non-converged* flexible mean over-estimates the variance by two- to four-fold, because the

un-converged mean leaves inflated residuals. The honest-residual family needs a converged mean.

The fix battery: a pre-registered ranking, refuted honestly. We pre-registered a single ranking of five ways to fit an input-dependent $\sigma(x)$ and tested it. The calibration criterion passed — the cross-fitted estimator's held-out standardised squared residual at $N = 1000$ was $[0.958, 0.930, 1.022]$ across seeds, all inside the acceptable band $[0.9, 1.1]$. The *ranking* criterion was **refuted** and is reported as such: the order depends on sample size. At $N = 200$ a robust training loss that down-weights the model's own variance in the gradient (a known small-sample fix) wins; a per-region recalibration is the *worst* arm, because coarse steps fit a smooth noise function badly. At $N = 1000$ the disease is gone and the four likelihood-based arms (all but per-region recalibration) are indistinguishable, falling within 0.054–0.063. There is no single winner; the honest statement is regime-dependence. Table 7 gives the numbers.

Table 7: The noise-flexibility fix battery. Noise-recovery error (lower is better) at two sample sizes. The pre-registered single ranking is refuted — at $N = 200$ the robust loss is best and recalibration clearly worst, but at $N = 1000$ the disease is gone and the four likelihood-based arms collapse into a narrow 0.054–0.063 band that is no longer meaningfully ranked, with per-region recalibration (0.079) still clearly worst.

Fix arm	Noise-recovery error, $N = 200$	Noise-recovery error, $N = 1000$
Robust down-weighted loss	0.139 (best)	0.054 (best)
Cross-fitted noise head	0.146	0.062
In-sample residual variance (mean fit first)	0.162	0.060
Standard joint likelihood (mean and variance together)	0.163	0.063
Per-region recalibration	0.280 (worst)	0.079 (worst)

The capacity read unifies with the count read — and the knee is coarse here too. Where the fix battery above held the noise model at full flexibility and varied the *estimation method*, the capacity read now varies the *flexibility of the noise model itself*, choosing among the four rungs v0–v3 with the same held-out knee. This gives the cleanest statement of the report's methodological theme. On the homoscedastic control — constant true noise — the knee abstains to the constant model on all three seeds: no false positive. On the heteroscedastic problem the knee detects that the noise varies (it leaves the constant model) but stops at the coarse three-step function, **not** at the smoother functions that recover the true noise curve markedly better. (Which smoother rung recovers it *best* is itself sample-size-dependent: at this sample size the smooth monotone rung wins on two seeds of three, and at four times the data the fully flexible rung wins on every seed — sensible, since the true noise curve is not exactly of the monotone rung's parametric form. The stable statement is that the rungs *above* the knee's choice recover the noise better, not that one particular rung is "the" correct class.) Figure 6 shows why this is not a failure of the instrument but a property of what held-out fit can see. The left panel is the ground-truth noise-recovery error by rung: it drops well below the three-step rung at the smoother rungs. The right panel is the held-out negative log-likelihood the knee actually reads (lower is better, so a drop is an improvement): it drops sharply — a large improvement — from the constant to the three-step rung, then is **flat within noise** across the three-step, smooth, and flexible rungs, the further improvement from the three-step

to the smooth rung being only about 0.003 nats, below the significance bar. The knee stops at the three-step rung because that is where the *held-out fit* stops improving significantly, even though the *noise recovery* keeps improving.

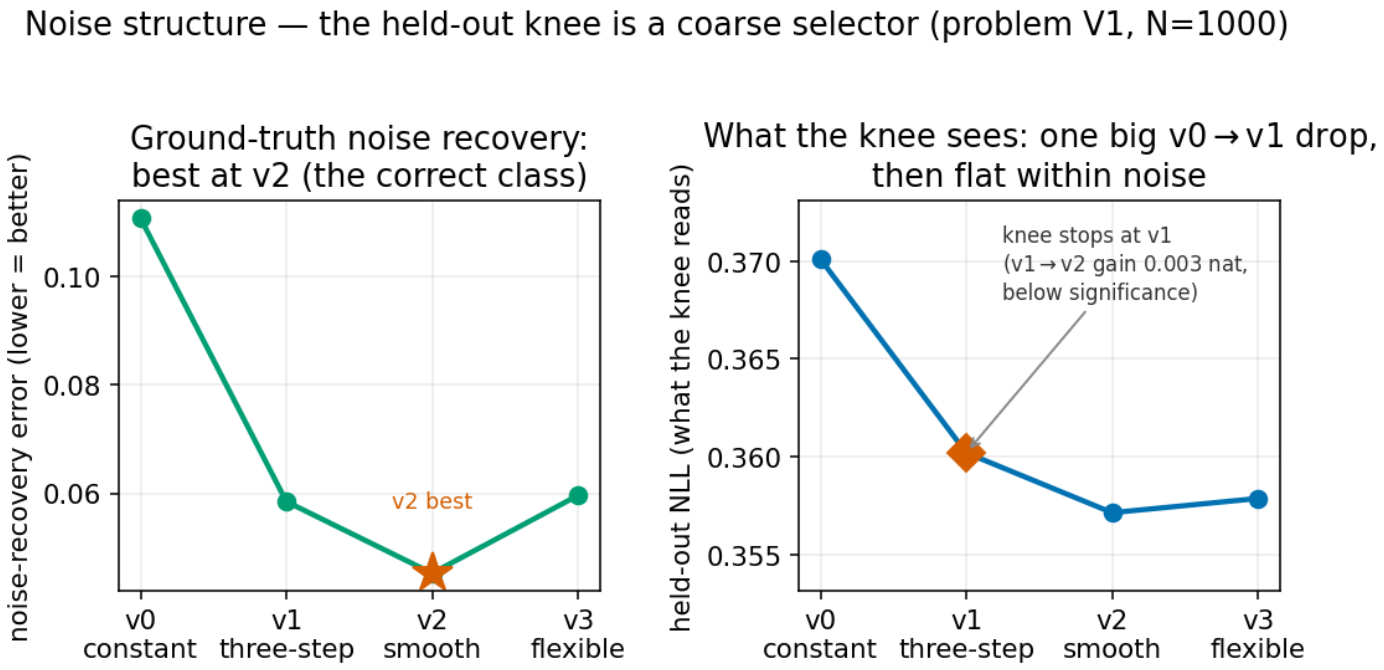

Figure 6: The held-out-NLL knee is a coarse noise selector. Left: the ground-truth noise-recovery error falls well below the three-step rung at the smoother rungs (which of those wins depends on sample size; see text). Right: the held-out fit the knee reads drops sharply to the three-step rung, then is flat within noise — so the knee stops at the three-step rung. Held-out fit cannot see the recovery gain that separates the smoother rungs from the three-step one.

Three checks confirm this is intrinsic, not small-sample noise. First, a known-answer test on synthetic data with a *strong* noise trend does resolve the smooth rung (the held-out gain there is +0.032 nats, ten times larger), so the instrument can read the smooth rung when the signal is there. Second, repeating the real read at four times the data does **not** move the knee to the smooth rung on any seed: the three-step-to-smooth held-out gain stays at 0.0006, 0.0000, 0.0026 nats — no larger than at the smaller sample. More data sharpens the *detection* of heteroscedasticity (the weakest seed crosses from the constant to the three-step rung) but not the *resolution* of the function class. Third, and decisively, the gap has an exact analytic size: for this noise curve, the population-level held-out gain of the *true* smooth noise function over the *best possible* three-step approximation to it is 0.0029 nats per example — a numeric integral whose three equal-mass input regions contribute 0.0075, 0.0012, and 0.0001 nats (averaged with weight one third each), the region around the curve's steepest point carrying nearly all of it. This is a hard ceiling that matches the measured increments (and the "0.003 nats" of Figure 6), so the flat surface is not estimation noise; there is genuinely almost nothing there for a fit-based reader to detect. This is a known blind spot of the log score, the scoring rule behind held-out fit: it is the only proper scoring rule that is *local* — it reads the predictive density only at the value that actually occurred (Gneiting and Raftery 2007) — and for any single residual one can construct a *different* noise level that assigns that residual exactly the same score (Camporeale and Carè 2021), so noise functions of visibly different shape can tie. This

is worth seeing quantitatively. Take the correct mean and predict the spread as $\sigma(1+\delta)$ in place of the true σ. A Gaussian with predicted spread $\tilde{\sigma}$ assigns an outcome y (with correct mean μ) the negative log-density $\frac{1}{2}\log(2\pi\tilde{\sigma}^2) + (y-\mu)^2/(2\tilde{\sigma}^2)$; averaging over residuals whose true spread is σ (so that $\mathbb{E}[(y-\mu)^2] = \sigma^2$) gives expected negative log-score $\frac{1}{2}\log(2\pi) + \log\tilde{\sigma} + \sigma^2/(2\tilde{\sigma}^2)$. Subtracting its value at the honest choice $\tilde{\sigma} = \sigma$ (which is $\frac{1}{2}\log(2\pi) + \log\sigma + \frac{1}{2}$), the constant and $\log(2\pi)$ terms cancel, and with $\tilde{\sigma} = \sigma(1+\delta)$ the expected log-score *lost* by mis-stating the spread is

$$\log(1+\delta) \;+\; \frac{1}{2(1+\delta)^2} \;-\; \frac{1}{2} \;=\; \delta^2 + O(\delta^3), \tag{7}$$

second order in the relative noise error δ (the second-order expansion follows because this expression and its first derivative both vanish at $\delta = 0$, while its second derivative there is 2) — whereas the noise-recovery error is $|\delta|$ itself, first order. A five-percent noise error costs only about 0.0025 nats of log score, which is why the two panels of Figure 6 can disagree so starkly. The faithful read of the noise-function class is the noise-recovery error, the analogue of the local advantage in Section 3 — though, unlike the local advantage, it requires knowing the true noise, so it is a validation instrument for studies like this one rather than a read deployable on real data; a deployable fine selector would need a *non-local* proper score (one that reads the whole predictive distribution, not just its value at the outcome), which is a queued refinement. The knee under-reports here in the same direction it did for component count, though less severely: here it at least detects that the noise varies and misses only the exact class, whereas for component count it missed the structure entirely.

6. From a measurement to a deployable selector

The three studies so far treat the held-out read as a *measurement instrument*: it tells us how much capacity a problem needs, region by region, and how faithfully each reader recovers it. A deployment question sits on top of that. Can a small, fixed map from input to capacity — a **selector** — be trained from those held-out reads and then shipped, so that at prediction time each input is served at the capacity it needs, with no held-out data in hand? Section 3 already showed the answer is yes in one case: a small input-to-count map, trained on the held-out reads with the component ladder frozen, matched or beat a single global count on all nine component-count cases. This section hardens that one result into a recipe, validates the recipe end to end on the actual model rather than on a frozen table of scores, tests how far it carries — across input dimension, and across sample size in both the count and the depth setting — and locates where it stops.

6.1 The selector recipe

A selector is a small map $\pi(x)$ from the input to a weight vector over the capacity rungs; the prediction is the weighted combination of the rungs' predictive distributions, and the score is the held-out fit of that combination,

$$\sum_i \log \sum_c \pi_c(x_i)\; p_c(y_i \mid x_i), \tag{8}$$

the same held-out log-score as everywhere else, now with input-dependent weights. The open choice is *what to train π to imitate*. We compared five constructions of the training target on the frozen component-count reads, each scored by the held-out fit of the resulting combination (reported as negative log-likelihood, so lower is better) over the nine structured cases. Table 8 gives the ranking.

Table 8: Five ways to build the selector's training target, scored by the held-out negative log-likelihood of the weighted combination they produce (lower is better), averaged over the nine structured component-count cases. The winner imitates the *prior-informed soft responsibilities* — the posterior weight each rung takes for each held-out point, computed with a mild per-region prior on the rungs. The hard knee label is worst, one more downstream confirmation of the knee's unfaithfulness.

Selector training target	Held-out NLL (mean over 9 cases)
Prior-informed soft responsibilities	0.6807 (best)
Soft responsibilities, with neighbour smoothing	0.6873
Soft responsibilities, no per-region prior	0.6874
Raw per-example best rung	0.6926
Hard per-input knee label	0.7131 (worst)

The winner imitates **prior-informed soft responsibilities** — for each held-out point, the posterior weight each rung takes for it (its *responsibility*), computed with a mild per-region prior that tilts the weights toward the counts typical of that input region. Decomposing the gap between arms into its ingredients — each an average paired difference over the nine cases, with a bootstrap standard error — isolates what carries the result. Adding the per-region prior is worth $+0.0067 \pm 0.0021$ nats (about three standard errors — load-bearing); making the target a soft weight rather than a single best rung is worth $+0.0052 \pm 0.0023$ (modest); neighbour smoothing is worth $+0.0001$ nats, roughly sixty times smaller than the prior and indistinguishable from nothing. The recipe is therefore compact: **imitate the prior-informed soft responsibilities, and do not bother smoothing.**

One check remained. Imitating a target is a means to an end, and the end is held-out fit — so why not train the selector *directly* on it, maximising the held-out fit of the combination with no imitation target at all? We ran that arm (training on the training half only, so the read stays honest, and letting gradients reach only the selector, never the frozen ladder). It **ties** the imitation recipe and does not beat it: mean held-out NLL 0.6811 against the recipe's 0.6807, a gap of 0.0004 nats — about a third of a standard error — and ten times as much training yields no win either. The principled objective confirms the recipe rather than replacing it; because it does not improve on the simpler imitation target, the imitation recipe stands as the recipe of record.

6.2 The recipe, validated on the model itself

Everything above was measured on frozen tables of scores — the rungs' held-out fits, precomputed. The deployment claim needs the recipe to work when the model is a single trained network, not a lookup table. The model here is a **classifier over k classes**: it assigns each input a probability over k classes, each class carries its own small regression head (a predicted mean and spread), and the prediction is the class-probability-weighted combination of those heads — the same weighted-combination object as above, now produced end to end by one network. Its capacity axis is the number of classes.

The recipe becomes a **two-phase** procedure. Phase one freezes the classifier and its regression heads, trained by the ordered-prefix schedule of Section 2.1 with the class-count selector held quiescent. Phase two distils a

small post-hoc selector — trained on the prior-informed soft responsibilities of Section 6.1 — on a held-out split *inside* the training data, and reads the weighted combination. We compared this two-phase model against three references: the model as shipped today, which trains the selector *jointly* with the heads; a standalone selector trained outside the network on the same target; and an oracle that sweeps a single fixed class count and keeps the best by held-out fit. Table 9 collects the outcome.

Table 9: The two-phase recipe run end to end on the classifier-over-classes model, against three references (negative log-likelihood, lower is better; three seeds each). It beats the model as shipped on the staircase, matches a standalone selector everywhere, and is never worse than an oracle single class count on the single-component control — while training its heads on half the data the shipping and oracle models see.

Comparison	Result
Two-phase vs jointly-trained shipping model, on the staircase	two-phase wins all 3 seeds by 0.078–0.100 nats
Post-hoc selector vs a standalone selector, all 9 cases	agree to within 0.0036 nats (9 of 9)
Two-phase vs oracle single class count, single-component control	two-phase never worse (better by 0.003–0.041)

Three findings. First, the two-phase model **beats the jointly-trained shipping model** on the staircase, on every seed. The mechanism is visible in the trained weights: the jointly-trained selector barely moves off its starting point — it puts about half its weight on the single-class rung and spreads the rest thinly, on every seed — while the two-phase selector actually routes. Decoupling head training from selector training avoids the joint model's failure to commit. Second, the selector distilled *inside* the network is **indistinguishable** from a standalone selector trained outside it on the same target — the recipe does not depend on where the selector lives. Third, on the single-component control the two-phase model is **never worse** than an oracle that picks the single best class count, and is better on all three seeds: it does not pay for its flexibility on data that needs none. It achieves all of this with its heads trained on half the data the shipping and oracle models see. The recipe of Section 6.1 is thus a validated selector for the actual model, not only for a frozen table. (One caveat on the control margin: the oracle's single-class rung is a constant spread that ignores the input, whereas the routed model's single-class rung still depends on it, which inflates one seed's margin; the direction of the result does not depend on it.)

6.3 How far the count mechanism carries: input dimension

The staircase lives in one input dimension; real problems do not. Does the per-input count read survive extra input coordinates that carry no count information — nuisance dimensions the neighbourhood read must see past? We embedded the staircase in a D-dimensional input (the count structure on one coordinate, the rest pure nuisance) and read it with a neighbourhood selector, measuring the held-out advantage of the routed combination over a single global one-class model — the multi-dimensional analogue of the local advantage of Section 3. A variance-matched single-component twin in the same dimension is the control. Table 10 gives the degradation with dimension.

Table 10: The count mechanism ported to D input dimensions — held-out advantage of the routed combination over a single global model (nats; three seeds). With one nuisance coordinate the mechanism is intact and the control invents nothing; by five dimensions the advantage is gone. A measured degradation, bracketed by an analytic ground truth and a clean control.

Input dimension	Advantage over global (mean nats)	Seeds significant	Control advantage (max)
2 (one nuisance coord)	+0.073	3 of 3 (6–9 SE)	-0.012 (invents nothing)
5	-0.012	0 of 3	—
10	-0.038	0 of 3	—

With a single nuisance coordinate the mechanism is intact: the routed model beats the global one by $+0.073$ nats on all three seeds at six-to-nine standard errors — a magnitude comparable to the one-dimensional staircase wins — and the variance-matched control manufactures no advantage (largest -0.012 nats, on the safe side of zero). By five dimensions the advantage is gone (null on all seeds), and it stays gone at ten. This is a *measured degradation*, not a failure: the study was built to find where the neighbourhood read breaks, and it breaks between two and five nuisance dimensions, with an analytic ground truth and a clean control bracketing it. One reading trap is worth flagging, because it is the same one Section 3 warned of. A raw *capture rate* of the routed count — the fraction of points routed to their true count — *rises* from two to five dimensions, which looks like improvement but is not: at five and ten dimensions the selector collapses to calling almost everything one class, which trivially "captures" the third of points that truly are one class, while the advantage is null. The faithful degradation readout is the advantage curve, not the capture rate — the knee-overshoots-count / advantage-is-faithful pattern of Section 3, reappearing in higher dimension.

6.4 Two power curves: where more data rescues the per-input read, and where it does not

Two of the three per-input reads were left qualified: the moving-mode count (Section 3, recovered on one seed of three) and the per-input depth read (Section 4, at the noise floor at 500 held-out points). Both qualifications were about *power* — too few held-out points per input region. The clean test of a power limit is a **power curve**: hold the instrument fixed, sweep the held-out sample size, and watch whether the per-input signal crosses its **detection floor** — the two-standard-error width of the advantage estimate (with the standard error corrected for the overlap between the many random data-splits it averages over), the bar an advantage must clear to count as real — and, crucially, whether that floor is *trustworthy*: whether the structure-free control stays *below* its own floor at the same sample size. We ran the identical power-curve apparatus on both lanes. They came out opposite.

The moving-mode count recovers with data. Sweeping the held-out sample size on the moving-mode problem (1000, 4000, then 16000 points), the recovered mid-region signal — the held-out advantage of the flexible fit over the single-component baseline in the input band where the two modes genuinely overlap — *grows monotonically*

with data. It clears the two-standard-error bar on two seeds of three already at the smallest size (two seeds strongly significant, one genuinely absent), and consolidates to all three seeds by four times the data, holding there at sixteen times. The variance-only control — humps in *spread*, never a genuine second mode — stays flat on all three seeds at *every* size, its mixture collapsing to a single component throughout. So the moving-mode negative of Section 3 was **under-powered, not unidentifiable**: with enough held-out data the read recovers, and the control never fires. Table 11 gives the per-seed curve.

Table 11: The moving-mode count power curve. Mid-region held-out advantage of the flexible fit over the single-component baseline (nats, per seed), against held-out sample size. The signal grows monotonically and consolidates from two of three seeds to all three; the variance-only control stays flat at every size. Recovery is driven by more data, not a different instrument.

Held-out points	Seed 0	Seed 1	Seed 2	Seeds recovered	Control
1000	+0.025	−0.004	+0.057	2 of 3	flat, 3 of 3
4000	+0.064	+0.087	+0.063	3 of 3	flat, 3 of 3
16000	+0.063	+0.090	+0.101	3 of 3	flat, 3 of 3

The per-input depth read does not. The same apparatus on the depth ladder (500, 2000, 8000 held-out points) tells the opposite story. The per-input depth advantage over a single global depth stays pinned near 0.001–0.0015 nats at *every* size — flat, not growing — while the detection floor falls by nearly an order of magnitude as data accrues (from 0.0121 to 0.0014 nats). At the largest size the structured problem crosses on two seeds of three — but so does the **structure-free control**, at the same size. Its spurious advantage (+0.0014 nats) is statistically indistinguishable from the structured one's (+0.0015; a two-sample test gives $t = 0.11$), and because the control's own floor has fallen further (0.0007 against the structured problem's 0.0014), the control in fact clears its floor by the *wider* margin. The instrument reports as much per-input depth structure in data built to have none as in the real problem, so it cannot separate them: the crossing is a floor dropping below a fixed offset, not a signal emerging. Table 12 gives both signals and both floors.

Table 12: The per-input depth power curve, for the structured problem and the structure-free control at each held-out sample size. Each "signal" is the mean per-input depth advantage over a single global depth (nats), with in parentheses how many of three seeds *cross* — have their own advantage exceed their own detection floor; each "floor" is that mean two-standard-error bar. Both signals stay flat while both floors shrink; at 8000 points *both* the real problem and the control cross, with indistinguishable signals ($+0.0015$ versus $+0.0014$, $t = 0.11$) and the control clearing its floor by the wider margin. The crossing is a floor-below-a-fixed-offset artifact, not a recovered depth signal — so the per-input depth read is *not* certified at any size here.

Held-out points	Structured signal (crosses)	Structured floor	Control signal (crosses)	Control floor
500	+0.0010 (0 of 3)	0.0121	+0.0019 (0 of 3)	0.0080
2000	+0.0004 (0 of 3)	0.0033	+0.0006 (0 of 3)	0.0022
8000	+0.0015 (2 of 3)	0.0014	+0.0014 (2 of 3)	0.0007

Same power-curve apparatus, opposite verdict: the count signal grows and its control stays flat; the depth signal is flat and its control crosses with it. That the two lanes diverge under identical treatment says the depth read's problem is not simply *less data than the count read had*. The next study asks whether it is a problem of a different *kind*.

6.5 Why the depth lane is different: representable is not learnable

Every per-input study needs a positive control — a problem where the structure is unarguably present, so that a null read convicts the *instrument* rather than the *data*. For component count the staircase is that control: its counts are true by construction. For depth we tried to build the analogue — a problem where the per-input depth requirement is *provable*. On half the input the target is linear (needs no depth); on the other half it is a function a shallow network provably cannot represent: a **tent map folded on itself five times**. The tent map is the triangle-shaped fold that sends the unit interval up then back down; composing it with itself k times produces 2^k oscillations, and a depth-separation theorem (Telgarsky 2016) shows a shallow network needs exponentially many units to match what a deep network represents with a handful — so the five-fold fold (thirty-two straight-line pieces) *demands* depth on that half of the input, by theorem.

The result is a clean **learnability-versus-representability** asymmetry. The deep-required half is representable at depth two and above — the theorem guarantees it — yet a network trained by gradient descent sits about 1.1–1.2 nats below the achievable fit at *every* trained depth, on every seed, and does not climb as depth grows; the easy linear half reaches its optimum. Suspecting an unlucky start, we retrained with eight independent random initialisations at each depth and kept the best by training fit: the deep-required half stays 1.17–1.21 nats below the achievable fit at every depth, and even the most generous possible read — best of eight scored on the test set itself — stays 1.14–1.17 nats short everywhere, never closing as depth increases. Table 13 collects

the gap. The provably-deep target is *representable but not gradient-learnable* at these scales.

Table 13: The learnability gap on the provably-deep target. The deep-required half sits more than a nat below the achievable fit at every trained depth and under every restart budget, and never closes with depth; the easy linear half reaches its optimum. Representable by theorem, not reachable by gradient descent.

Read of the deep-required half	Gap below achievable fit (nats)
Single fit, deepest network (3 seeds)	+1.16, +1.12, +1.07
Best of 8 restarts, kept by training fit (all depths)	1.17–1.21
Best of 8 restarts, scored on test (all depths)	1.14–1.17
Easy linear half, for contrast	≤ 0.05

This reframes Section 6.4's depth null. The depth lane has **no learnable positive control**: the one target for which the depth requirement is provable cannot be fit by the optimiser at all, so it cannot serve as the presence-of-structure anchor the count staircase provides. The flat per-input depth read is therefore consistent with two distinct explanations — genuinely no per-input depth structure in the *reachable* problems, or a real structure the optimiser cannot install — and the present studies cannot separate them, because the discriminating problem is unlearnable. Whether *any* target that both requires depth and is learnable exists is an open question. Two ways forward remain: a bounded search for such a target, or recording the learnability-versus-representability asymmetry as the depth-lane finding and closing the per-input depth question as a compute-saving story rather than a recovery one. Which to take is a research-priority decision left open here; the per-input depth result does not need it resolved to stand as reported — a null with a named, honest cause rather than an unexplained one.

7. What holds, what does not, and where the boundary is

What holds. A single held-out **reader** — one nested model (or, for noise, an ordered family of increasing flexibility), scored on fresh data — selects capacity across three unrelated axes: mixture-component count, network depth, and noise-model flexibility. In each, a held-out read *abstains on its control*: single-component data, uniform-depth data, and constant-noise data each read negative, so the instrument does not manufacture structure. And in each, a read *discriminates* genuine structure from that control — but which read does so differs by axis. For depth the discriminator is the aggregate knee read on dedicated same-depth models (depth 2 on both structured problems versus abstention on the control); for noise it is the aggregate knee's detection (heteroscedasticity detected versus the constant-noise control). For component count the aggregate knee under-reads even on structured data, so the discriminator is the *local advantage* — large on the staircase (up to 0.20 nats) and near-zero on the single-component controls. And the count read is not only a measurement. Distilled into a small input-to-capacity map — a **selector** trained on the held-out reads with the ladder frozen — it ships: it matches or beats a single global count on every component-count case, and, run end to end on the classifier-over-classes model, it beats the jointly-trained model on the staircase and never underperforms an oracle single count on the control (Section 6).

What is qualified. The *per-input* read — how capacity varies input to input — is data-hungry, and how much data rescues it depends on the setting. For component count it is recovered clearly on fixed-mode structure at

the sample sizes here, and the one moving-mode case that failed at those sizes is recovered by more held-out data — the failure was under-powering, not unidentifiability (Section 6.4) — though the read survives only about one nuisance input dimension before it degrades (Section 6.3). For depth the per-input read is not established at any sample size tried, up to sixteen times the base size: the signal stays flat while the detection floor shrinks, and the structure-free control crosses alongside the real problem, so no depth signal is certified (Section 6.4). For noise structure the read comes through only in aggregate. The honest headline is: aggregate capacity selection is robust; per-input capacity is recoverable but data-hungry for component count, and — for depth — not established at all here, for a reason Section 6.5 makes precise.

The unfaithful knee. The obvious reader — the point at which held-out fit stops improving — systematically mis-reports, per input in every setting and at the aggregate level in every setting too. Per input it is noisy where neighbourhoods are small (component count, depth). At the aggregate level it fails once for each of three distinct reasons, and together they map the reader's failure surface: it is *blind* where held-out fit is genuinely flat across model classes of different quality (noise structure — a property of the log score's locality, Section 5); it is *biased by the ladder itself* wherever the coherence cost of nesting changes between rungs, under-reading where the cost peaks mid-ladder (component count, Section 3) and over-reading where the cost decays from the bottom rung (depth, Section 4). The faithful read is always a contrast against a reference outside the ladder's own increments: the local advantage over the single-component model for count, the dedicated same-depth comparison for depth, the direct recovery error against the truth for noise. Reading capacity means reading that contrast, not the knee.

The identifiability boundary. The moving-mode count — a count that rises then falls across the input — looked at first like a genuine negative, recovered on one seed of three where the fixed staircase was recovered on all three. It is not. A sample-size sweep recovers it with more held-out data, on all three seeds by four times the sample, while the control stays flat throughout; the non-nested reader is equally seed-fragile at the small size; so the difficulty was power, not identifiability, and not specific to nesting (Section 6.4). The real boundaries lie elsewhere. One is **dimension**: the per-input count read survives roughly a single nuisance input coordinate and is gone by five, a measured degradation bracketed by an analytic ground truth and a clean control (Section 6.3). The other, sharper one is the per-input **depth** read, null at every sample size tried. Section 6.5 supplies the reason it may be null in principle and not only for want of data: the one target for which a per-input depth requirement is *provable* — a function a shallow network cannot represent, by a depth-separation theorem — turns out to be representable but not learnable by gradient descent, so the depth lane has no learnable positive control to anchor a per-input read against. Whether any learnable depth-requiring target exists is open; resolving it — a bounded search for one, or recording the learnability-versus-representability asymmetry as the finding and treating per-input depth as a compute-saving story — is left as future work and changes nothing reported here.

Cost. Nesting is not free, and its cost is not one number. On the component ladder it is small (0.13 nats at worst), concentrated on middle rungs, and never touches the top rung; on the depth ladder it is large at the shallow end (up to 0.72 nats at depth 1), touches every rung, and is nearly absent only on the control. In both cases the *profile* of the cost across rungs — not just its size — matters, because the knee inherits the profile's slope (see above). Any deployment claim must carry both the cost and its shape.

Positioning. The nearest published method is hierarchical stacking (Yao et al. 2022), which makes stacking weights depend on the input and is most useful exactly when predictive performance is heterogeneous across inputs. It reads input-dependent *averaging weights over a fixed set of models*. What is new here is reading an input-dependent *capacity over a nested ladder* — a monotone complexity index rather than a categorical model average

— together with the three results that qualify it: that the naive reader is unfaithful (per input everywhere, and at the aggregate level through the log score's locality and through the ladder's own coherence-cost profile), that the per-input read is power-limited while the aggregate read is robust, and that the nesting cost has a shape the reader inherits. The ordered-prefix training is the nested-dropout scheme of Rippel, Gelbart, and Adams (2014); the global read is the stacking of Yao et al. (2018); the variance-collapse diagnosis connects to the faithful-regression line of Stirn et al. (2023). The separate question of how this framework relates to mixture-of-experts routing is treated in a companion note (*Two Answers to Per-Input Capacity: Sparse Mixture-of-Experts Routing versus a Held-Out Capacity Ladder*, 2026, distributed alongside this report) and is not absorbed here.

Deployment recommendations, by setting. For a global noise level with a linear mean, the marginal-likelihood estimator; for a global noise level with a flexible mean, the cross-fitted estimator provided the mean converges; for an input-dependent noise level at small samples, the robust down-weighted loss; for calibration at moderate samples, the cross-fitted noise head; for distribution-free intervals, a conformal wrapper — a wrapper that sets interval widths from held-out errors to hit a target coverage rate with no distributional assumptions. For per-input capacity, read the faithful contrast, not the knee, and carry the power caveat; and to *ship* it, distil the prior-informed soft responsibilities into a post-hoc selector on a held-out split (Section 6.2) — the recipe that matched or beat both the jointly-trained model and a fixed oracle count, while smoothing and a direct held-out objective each added nothing.

Appendix A. The maximum-likelihood variance bias

Take a linear mean, $y_i = x_i^\top \beta + \varepsilon_i$ with ε_i independent, mean zero, variance σ^2, and N observations with a p-dimensional β. (In this appendix p counts mean parameters — distinct from the predictive densities p_c of Section 2 — and H is a matrix, unrelated to the toy problem H of Section 4.) Stack the inputs into the $N \times p$ **design matrix** X (row i is $x_i^\top$) and the targets into the vector y. The least-squares fit is $\hat{\beta} = (X^\top X)^{-1} X^\top y$ and the fitted residuals are $\hat{\varepsilon} = y - X\hat{\beta} = (I - H)\varepsilon$, where $H = X(X^\top X)^{-1} X^\top$ is the projection onto the column space of X (the step uses $HX = X$: the projection leaves the columns of X fixed, so the mean part cancels and only the projected noise remains). The maximum-likelihood variance is the mean squared fitted residual, $\hat{\sigma}^2 = \frac{1}{N}\,\hat{\varepsilon}^\top \hat{\varepsilon}$. Using $\hat{\varepsilon} = (I - H)\varepsilon$ and that $I - H$ is a projection ($(I - H)^2 = I - H$),

$$\mathbb{E}\big[\hat{\sigma}^2\big] = \tfrac{1}{N}\,\mathbb{E}\big[\varepsilon^\top (I - H)\varepsilon\big] = \tfrac{1}{N}\,\sigma^2\,\operatorname{tr}(I - H) = \tfrac{1}{N}\,\sigma^2\,(N - p) = \frac{N - p}{N}\,\sigma^2. \tag{9}$$

The trace step uses $\mathbb{E}[\varepsilon^\top A \varepsilon] = \sigma^2 \operatorname{tr}(A)$ for independent errors, and $\operatorname{tr}(I - H) = N - \operatorname{tr}(H) = N - p$ because H projects onto a p-dimensional space. The estimate is biased low by the fraction of directions the mean consumed, p/N. A flexible mean spends far more effective directions than a linear one, so its residuals shrink further and its variance collapses further — the finite-sample mechanism of Section 5, in closed form for the linear case.

Appendix B. The stacking objective

Given the held-out score table $\{s_i(c)\}$, stacking maximises the held-out log-score of the mixture over the $(C - 1)$-simplex $\mathcal{S} = \{\pi : \pi_c \geq 0,\ \sum_c \pi_c = 1\}$ (the set of valid weight vectors):

$$\hat{\pi} = \arg\max_{\pi \in \mathcal{S}}\ \mathcal{L}(\pi), \qquad \mathcal{L}(\pi) = \sum_i \log \sum_c \pi_c\, e^{\,s_i(c)}. \tag{10}$$

$\mathcal{L}$ is concave in π: each term $\log \sum_c \pi_c e^{s_i(c)}$ is the log of a linear function of π, hence concave, and a sum of concave functions is concave. The maximiser is found by a short fixed-point iteration that sets each component's weight to the **average responsibility** it takes for the held-out points,

$$\pi_c \leftarrow \frac{1}{N}\sum_i r_{ic}, \qquad r_{ic} = \frac{\pi_c\, e^{s_i(c)}}{\sum_{c'} \pi_{c'}\, e^{s_i(c')}}, \tag{11}$$

where r_{ic} is the responsibility component c takes for held-out point i. This is the expectation-maximisation update for a mixture-weight problem: each pass is a monotone ascent step on the concave objective $\mathcal{L}$ — it never decreases it, by the standard expectation-maximisation argument (each pass maximises a lower bound on $\mathcal{L}$, obtained from Jensen's inequality, that touches $\mathcal{L}$ at the current π) — so the iteration climbs to the unique maximiser in a handful of passes. It is the right objective because the log-score is a strictly proper scoring rule: its expectation is maximised only by the true predictive distribution, so weights that score well on held-out data are weights that describe the data-generating process, and capacity that overfits — scoring poorly out of sample — is charged automatically, with no penalty term.

References

Camporeale, Enrico, and Algo Carè. 2021. "ACCRUE: Accurate and Reliable Uncertainty Estimate in Deterministic Models." *International Journal for Uncertainty Quantification* 11 (4).

Gneiting, Tilmann, and Adrian E. Raftery. 2007. "Strictly Proper Scoring Rules, Prediction, and Estimation." *Journal of the American Statistical Association* 102 (477): 359–78.

Rippel, Oren, Michael A. Gelbart, and Ryan P. Adams. 2014. "Learning Ordered Representations with Nested Dropout." In *Proceedings of the 31st International Conference on Machine Learning (ICML)*, 32:1746–54. PMLR.

Sivula, Tuomas, Måns Magnusson, Asael Alonzo Matamoros, and Aki Vehtari. 2020. "Uncertainty in Bayesian Leave-One-Out Cross-Validation Based Model Comparison." *arXiv Preprint*.

Stirn, Andrew, Harm Wessels, Megan Schertzer, Laura Pereira, Francisco J. Sanchez-Rivera, and David A. Knowles. 2023. "Faithful Heteroscedastic Regression with Neural Networks." In *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 206:5593–613. PMLR.

Telgarsky, Matus. 2016. "Benefits of Depth in Neural Networks." In *Proceedings of the 29th Annual Conference on Learning Theory (COLT)*, 49:1517–39. PMLR.

Yao, Yuling, Gregor Pirš, Aki Vehtari, and Andrew Gelman. 2022. "Bayesian Hierarchical Stacking: Some Models Are (Somewhere) Useful." *Bayesian Analysis* 17 (4): 1043–71.

Yao, Yuling, Aki Vehtari, Daniel Simpson, and Andrew Gelman. 2018. "Using Stacking to Average Bayesian Predictive Distributions (with Discussion)." *Bayesian Analysis* 13 (3): 917–1007.